

Data Structures Notes: Abstract Data Type (ADT) and Queues

1. Abstract Data Type (ADT)

An **Abstract Data Type (ADT)** is a logical description of a data type that specifies:

- **What data is stored**
- **What operations can be performed**
- **What those operations do**

It does **not** specify how the operations are implemented.

Example: Queue ADT

A Queue ADT specifies operations such as:

- `enqueue ()` – insert an element
- `dequeue ()` – delete an element
- `peek ()` – view the front element
- `isEmpty ()` – check whether the queue is empty
- `isFull ()` – check whether the queue is full

The implementation may use an **array** or a **linked list**.

ADT vs Data Structure

ADT	Data Structure
Logical model	Physical implementation
Describes what operations do	Describes how operations are implemented
Independent of implementation	Depends on implementation
Example: Queue ADT	Array Queue, Linked Queue

2. Queue

A **Queue** is a linear data structure in which insertion takes place at one end called **REAR**, while deletion takes place at the other end called **FRONT**.

Queue follows:

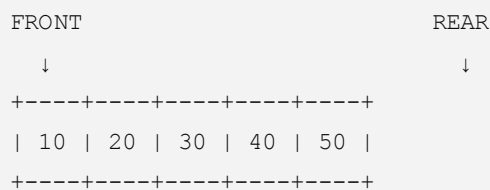
FIFO – First In, First Out

The element inserted first is removed first.

Real-life examples

- People standing in a queue
- Printer job scheduling
- CPU process scheduling
- Network packet processing
- Customer service systems

Basic representation



- 10 is the first element to be removed.
- New elements are inserted after 50 .

3. Basic Queue Operations

Operation	Description
Enqueue	Insert an element
Dequeue	Delete an element
Peek/Front	View front element
Rear	View last element
isEmpty	Check whether queue is empty
isFull	Check whether queue is full

Queue conditions

Queue Empty:

```
FRONT == -1
```

or, depending on implementation:

```
FRONT > REAR
```

Queue Full in a simple array queue:

```
REAR == MAX - 1
```

4. Types of Queue

The major types are:

1. **Simple Queue / Linear Queue**
2. **Circular Queue**
3. **Priority Queue**
4. **Deque (Double Ended Queue)**

For this topic, we focus on the first three.

5. Simple Queue / Linear Queue

A **Simple Queue** is a queue in which:

- Insertion is performed at the **REAR**
- Deletion is performed at the **FRONT**
- It follows **FIFO**

Representation

```
FRONT                                REAR
  ↓                                ↓
+---+---+---+---+---+
| 10 | 20 | 30 | 40 | 50 |
+---+---+---+---+---+
```

5.1 Enqueue Operation

Algorithm: ENQUEUE

Input: ITEM

Output: ITEM inserted into queue

```
Algorithm ENQUEUE (ITEM)

1. If REAR == MAX - 1
    Print "Queue Overflow"
    Return

2. If FRONT == -1
    FRONT = 0

3. REAR = REAR + 1

4. QUEUE[REAR] = ITEM

5. Return
```

Example

Initially:

```
FRONT = -1
REAR  = -1
```

Insert 10 :

```
FRONT = 0
REAR  = 0

+-----+
| 10 |
+-----+
```

Insert 20 :

```
FRONT = 0
REAR  = 1

+-----+-----+
| 10 | 20 |
+-----+-----+
```

5.2 Dequeue Operation

Algorithm: DEQUEUE

Output: Deleted element

```
Algorithm DEQUEUE()  
  
1. If FRONT == -1 OR FRONT > REAR  
    Print "Queue Underflow"  
    Return  
  
2. ITEM = QUEUE[FRONT]  
  
3. If FRONT == REAR  
    FRONT = -1  
    REAR = -1  
Else  
    FRONT = FRONT + 1  
  
4. Return ITEM
```

Example

Before deletion:

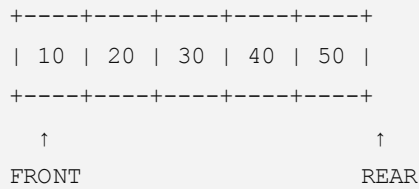
FRONT		REAR
↓		↓
+-----+-----+-----+		
10 20 30		
+-----+-----+-----+		

After deleting 10 :

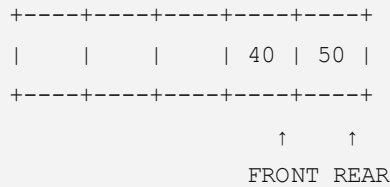
FRONT		REAR
↓		↓
+-----+-----+-----+		
10 20 30		
+-----+-----+-----+		
↑		
deleted		

5.3 Problem with Simple Queue

Consider an array of size 5:



After deleting 10, 20, and 30 :



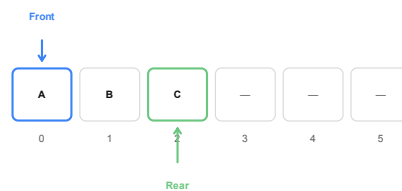
Although positions 0, 1 and 2 are empty, REAR is already at the last position.

Therefore, another insertion may report:

Queue Overflow

This is called **false overflow** or **space wastage**.

The solution is the **Circular Queue**.



Linear queue: 3 items, front 0 → rear 2.

Linear

Circular

Enqueue D

Dequeue A

Reset

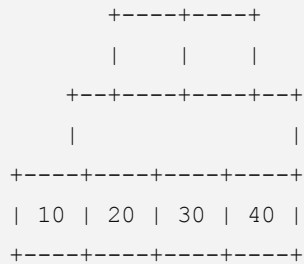
[Give feedback](#)

6. Circular Queue

A **Circular Queue** is a queue in which the last position is logically connected to the first position.

It efficiently utilizes the unused spaces created by deletion.

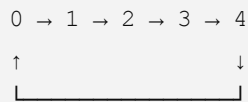
Concept



The next position after the last index is the first index.

Circular movement

If array size is 5 :



The next position is calculated using:

$$(\text{REAR} + 1) \% \text{MAX}$$

7. Circular Queue Conditions

Empty condition

A common implementation uses:

$$\text{FRONT} == -1$$

Full condition

$$(\text{FRONT} == 0 \ \&\& \ \text{REAR} == \text{MAX} - 1)$$

OR

```
(FRONT == REAR + 1)
```

An equivalent and commonly used form is:

```
(REAR + 1) % MAX == FRONT
```

8. Circular Queue Enqueue

Algorithm

```
Algorithm CIRCULAR_ENQUEUE(ITEM)
```

```
1. If (REAR + 1) % MAX == FRONT
    Print "Queue Overflow"
    Return
```

```
2. If FRONT == -1
    FRONT = 0
    REAR = 0
Else
    REAR = (REAR + 1) % MAX
```

```
3. QUEUE[REAR] = ITEM
```

```
4. Return
```

Example

Suppose:

```
MAX = 5
```

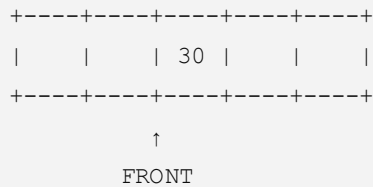
Initially:

```
FRONT = 0
```

```
REAR = 2
```

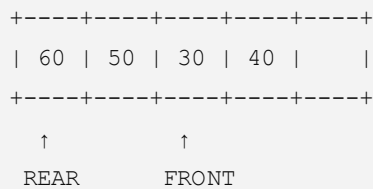
```
+-----+-----+-----+-----+
| 10 | 20 | 30 |   |   |   |
+-----+-----+-----+-----+
      ↑           ↑
    FRONT       REAR
```

Delete 10 and 20.



Now insert 40 , 50 , 60 .

The rear wraps around:



Thus, circular queue makes effective use of available space.

9. Circular Queue Dequeue

Algorithm

```
Algorithm CIRCULAR_DEQUEUE()

1. If FRONT == -1
    Print "Queue Underflow"
    Return

2. ITEM = QUEUE[FRONT]

3. If FRONT == REAR
    FRONT = -1
    REAR = -1
Else
    FRONT = (FRONT + 1) % MAX

4. Return ITEM
```

10. Advantages of Circular Queue

- Better utilization of memory
- Avoids false overflow
- Efficient insertion and deletion

- Suitable for buffers
- Useful in CPU scheduling and networking

Disadvantage

The implementation is slightly more complex because the `FRONT` and `REAR` positions must wrap around.

11. Priority Queue

A **Priority Queue** is a queue in which every element is associated with a **priority**.

Deletion is based on priority rather than simply the order of insertion.

Example

Suppose:

Element	Priority
A	3
B	1
C	2

If a **smaller number means higher priority**, the deletion order is:

B → C → A

Real-life examples

- CPU process scheduling
 - Hospital emergency systems
 - Network packet scheduling
 - Printer scheduling
 - Operating system task management
-

12. Types of Priority Queue

1. Ascending Priority Queue

The element with the **smallest priority value** is removed first.

Example:

Priority: 1 → 2 → 3 → 4

2. Descending Priority Queue

The element with the **largest priority value** is removed first.

Example:

Priority: 4 → 3 → 2 → 1

The meaning of a "high" priority number is implementation-dependent. The important point is that deletion follows priority order.

13. Priority Queue Representation

Consider:

Element	Priority
A	3
B	1
C	2
D	4

For ascending priority:

B(1) → C(2) → A(3) → D(4)

Deletion sequence:

B → C → A → D

If two elements have the same priority, a common rule is **FIFO among equal-priority elements**.

14. Priority Queue Enqueue

One simple array implementation maintains elements in sorted priority order.

Algorithm

```
Algorithm PRIORITY_ENQUEUE (ITEM, PRIORITY)
```

1. If queue is full
 Print "Queue Overflow"
 Return
2. If queue is empty
 Insert ITEM
 Return
3. Find the correct position according to PRIORITY.
4. Shift lower-priority elements one position to the right.
5. Insert ITEM at the correct position.
6. Return

Example

Existing queue:

A(1) → C(3) → D(4)

Insert:

B(2)

Result:

A(1) → B(2) → C(3) → D(4)

15. Priority Queue Dequeue

If the queue is maintained in sorted order, the highest-priority element is at the front.

Algorithm

```
Algorithm PRIORITY_DEQUEUE()
```

1. If queue is empty
 Print "Queue Underflow"
 Return

```
2. ITEM = QUEUE[FRONT]

3. FRONT = FRONT + 1

4. Return ITEM
```

For a descending-priority queue, the ordering is maintained in the opposite direction.

16. Queue Operation Analysis

Let:

- n = number of elements
- N = size of array

Simple Queue

Operation	Time Complexity
Enqueue	$O(1)$
Dequeue	$O(1)$
Peek	$O(1)$
isEmpty	$O(1)$
isFull	$O(1)$

The basic operations are constant time.

Circular Queue

Operation	Time Complexity
Enqueue	$O(1)$
Dequeue	$O(1)$
Peek	$O(1)$
isEmpty	$O(1)$
isFull	$O(1)$

The modulo operation used for wrapping does not change the asymptotic complexity.

Priority Queue

Complexity depends on implementation.

Unsorted Array

Operation	Time
Enqueue	$O(1)$
Find/Delete highest priority	$O(n)$

Sorted Array

Operation	Time
Enqueue	$O(n)$
Dequeue	$O(1)$

Therefore, the implementation determines the performance.

17. Comparison of Queue Types

Feature	Simple Queue	Circular Queue	Priority Queue
Principle	FIFO	FIFO	Priority
Insertion	Rear	Rear	Based on priority
Deletion	Front	Front	Highest/lowest priority
Memory utilization	May waste space	Efficient	Depends on implementation
Overflow problem	False overflow possible	Avoided	Depends on implementation
Enqueue	$O(1)$	$O(1)$	$O(1)$ or $O(n)$
Dequeue	$O(1)$	$O(1)$	$O(1)$ or $O(n)$

Feature	Simple Queue	Circular Queue	Priority Queue
Main application	Basic scheduling	Buffers, networking	CPU/emergency scheduling

18. Queue Overflow and Underflow

Queue Overflow

Occurs when an insertion is attempted on a full queue.

```
Queue is Full
  ↓
ENQUEUE
  ↓
OVERFLOW
```

Queue Underflow

Occurs when deletion is attempted on an empty queue.

```
Queue is Empty
  ↓
DEQUEUE
  ↓
UNDERFLOW
```

19. Important Applications of Queues

Simple Queue

- Printer queue
- Customer service
- CPU process management
- Keyboard input buffering

Circular Queue

- CPU round-robin scheduling
- Circular buffers
- Network data buffering
- Streaming data

- Memory management

Priority Queue

- CPU scheduling
- Dijkstra's algorithm
- Emergency room management
- Network routing
- Event simulation

20. Key Points for Examination

1. **Queue follows FIFO.**
2. **Insertion takes place at REAR.**
3. **Deletion takes place at FRONT.**
4. **Simple queue may suffer from false overflow.**
5. **Circular queue solves the space-wastage problem.**
6. Circular queue uses:

$(\text{REAR} + 1) \% \text{MAX}$

to move the rear circularly.

7. **Priority queue removes elements according to priority.**
8. In a priority queue, FIFO is generally applied among elements having equal priority.
9. **ADT describes what operations do, not how they are implemented.**
10. Array and linked list are implementation techniques for queue ADTs.

Quick memory trick

SIMPLE QUEUE

FIFO

Front → Delete

Rear → Insert

CIRCULAR QUEUE

FIFO + Wrap Around

Better memory utilization

PRIORITY QUEUE

Priority decides deletion

